

Unidad	Unidad 4
Entrega	Método de entrega de la actividad en el campus virtual

1. Enunciado

El objetivo de esta práctica es ampliar el sistema desarrollado en la Práctica 1 incorporando dos nuevas estructuras de datos: **grafos** y **tablas hash**. Partiendo del sistema de eventos basado en árboles binarios, ahora se debe modelar el **mapa del mundo del juego** mediante un grafo y la **gestión del inventario y estados del jugador** mediante tablas hash.

El mapa del juego se representará como un **grafo** donde:

- cada **nodo (vértice)** representa una localización del mundo
- cada **arista** representa un camino o conexión entre localizaciones
- el **peso de la arista** representa la distancia o coste (energía) de recorrerlo
- cada localización puede contener **eventos** (árboles binarios de la Práctica 1)

El inventario y los estados del jugador se gestionarán mediante **tablas hash** para permitir un acceso eficiente a los objetos recogidos, las localizaciones visitadas y los eventos ya completados.

Esta práctica forma parte del proyecto integrado que continuará en la siguiente entrega:

- **Práctica 3 (final):** integración completa de árboles, grafos y tablas hash con presentación del sistema

2. Descripción del sistema

Se debe ampliar en Java el sistema de aventura desarrollado en la Práctica 1 añadiendo dos nuevos módulos:

- **Módulo Mapa:** grafo que representa el mundo del juego y permite al jugador desplazarse entre localizaciones.
- **Módulo Inventario y Estados:** tablas hash para gestionar los objetos del jugador, las localizaciones visitadas y los eventos completados.

Ejemplo de mapa:

- el jugador comienza en la localización "Aldea inicial"
- desde ahí puede desplazarse al "Bosque", al "Río" o al "Mercado"
- cada desplazamiento consume energía (peso de la arista)
- al llegar a una localización, puede ejecutarse un evento (árbol binario)

Durante la partida, el estado del jugador se actualiza:

- el inventario crece o se modifica con los objetos obtenidos en los eventos
- se registran las localizaciones visitadas
- se registran los eventos ya completados para evitar repetirlos

2.1. Estructura mínima del programa

El programa debe implementarse usando programación orientada a objetos y deberá reutilizar las clases desarrolladas en la Práctica 1 (Jugador, Decision, Nodo, ArbolDecisionDinamico, etc.).

Se recomienda la siguiente estructura de clases adicional:

Clase Localizacion: Atributos mínimos

- id
- nombre
- descripcion
- arbolEvento (referencia al ArbolDecisionDinamico asociado, si existe)
- visitada (booleano)

Clase Arista: Atributos mínimos

- origen (Localizacion)
- destino (Localizacion)
- peso (coste de energía del desplazamiento)

Clase Mapa (Grafo): Implementación del grafo del mundo del juego.

Se debe implementar mediante al menos una de estas dos representaciones (se valorará implementar ambas):

- **Matriz de adyacencia**
- **Lista de adyacencia**

Debe permitir:

- añadir localización (vértice)
- añadir camino (arista) con peso
- comprobar si dos localizaciones están conectadas
- obtener localizaciones vecinas de una dada
- eliminar arista
- recorrido en profundidad (DFS)
- recorrido en anchura (BFS)
- camino mínimo entre dos localizaciones (algoritmo de Dijkstra)

Clase Inventario: Implementación mediante tabla hash.

La tabla hash debe implementarse manualmente (no se permite usar directamente HashMap de Java salvo con justificación previa). Debe resolver colisiones mediante alguna técnica estudiada:

- encadenamiento separado (listas enlazadas en cada posición), o
- direccionamiento abierto (sondeo lineal, cuadrático o doble hash)

Debe permitir:

- añadir objeto
- eliminar objeto
- buscar objeto por clave (id o nombre)
- comprobar si contiene un objeto
- mostrar todos los objetos
- obtener el número de objetos
- calcular el factor de carga

Clase RegistroPartida: También implementada mediante tabla(s) hash.

Debe gestionar:

- localizaciones visitadas
- eventos completados
- consulta eficiente de si una localización o evento ya fue visitado/completado

2.2. Funcionalidades obligatorias

El programa debe implementar como mínimo las siguientes operaciones:

Gestión del grafo (mapa)

- insertar localización
- insertar camino con peso
- eliminar camino
- consultar vecinos de una localización
- comprobar conectividad del mapa

Recorridos y caminos del grafo

- recorrido en profundidad (DFS) desde una localización
- recorrido en anchura (BFS) desde una localización
- camino mínimo entre dos localizaciones (Dijkstra)
- mostrar el coste total en energía del camino elegido

Gestión de la tabla hash (inventario)

- insertar, buscar y eliminar objetos
- mostrar el contenido de la tabla hash
- mostrar estadísticas: tamaño, factor de carga y número de colisiones

Simulación de partida

El programa debe permitir ejecutar una partida completa en la que:

- el jugador parte de una localización inicial
- se muestra el mapa (o las localizaciones accesibles)
- el jugador elige a qué localización desplazarse
- al moverse se descuenta la energía correspondiente al peso de la arista
- al llegar a la localización, si hay un evento (árbol de la Práctica 1) se ejecuta
- los objetos obtenidos se añaden al inventario (tabla hash)
- se registra la localización como visitada y el evento como completado
- la partida termina cuando el jugador llega a una localización objetivo o cuando pierde toda la vida/energía

Debe mostrarse al finalizar:

- ruta seguida por el jugador
- decisiones tomadas en cada evento
- estado final del jugador (vida, energía, oro)
- contenido del inventario
- camino mínimo alternativo que podría haber tomado (comparativa con Dijkstra)

3. Detalles de la entrega

- Código fuente, proyecto Java completo. Método de compresión ZIP (NO rar)
- El proyecto debe integrar el código de la Práctica 1 (reutilizando los árboles de decisión como eventos del mapa)
- Memoria breve (máx 4 páginas): explicación de la estructura del programa, descripción del mapa diseñado, justificación de la representación del grafo elegida, explicación de la función hash y del mecanismo de resolución de colisiones, y comparación de eficiencia entre las operaciones sobre el grafo y la tabla hash

3.1. Restricciones

No se aceptarán entregas que:

- no compilen
- Solo se evaluarán las prácticas teniendo en cuenta que solo pueden usar Java básico, sin estructuras como HashMap, ArrayList, LinkedList, etc.
- no utilicen realmente grafos (por ejemplo, simulando el mapa con condicionales o con una lista plana de destinos)
- no implementen una tabla hash propia (salvo autorización explícita del profesor)
- no integren la Práctica 1 (el mapa debe contener al menos un evento representado con árbol binario)
- tengan menos de 6 localizaciones y 8 caminos en el mapa
- tengan menos de 3 eventos asociados a localizaciones

4. Rubrica

Criterio	Peso
Implementación del grafo (mapa)	25%
Implementación de la tabla hash (inventario)	25%
Recorridos y algoritmo de camino mínimo	15%
Integración con la Práctica 1 (eventos)	15%
Simulación de partida	10%
Calidad del código y memoria	10%
Implementación del grafo (mapa)	25%